

RECONSTRUCTION_ROADMAP.md — Roteiro de Reconstrução

Propósito: documento vivo que registra o que já foi executado, o que está pendente e como cada etapa será abordada. Destinado a **agentes de IA e desenvolvedores** que precisem auditar, continuar ou revisar este projeto. Pode ser excluído após a conclusão total.

Última atualização: 21 de julho de 2026 — após conclusão do Bloco 2.

Visão geral do projeto

Campo	Valor
Nome	Born to Run — Treinamento e Saúde
Fundação	2015, Descalvado-SP
Público	~200 atletas corredores
Treinador	Robson Alves — Educador Físico, CREF 119911-G/SP, especialização meio fundo/fundo, Treinador nível 1 World Athletics
Instagram	@equipeborntorun
Stack	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, Supabase (Auth + Postgres + Storage)
Branch de trabalho	abacus-fable-rebuild

Organização em blocos

O plano original (`FABLE_REBUILD_PLAN.md`) definiu 16 fases técnicas. Para execução prática, agrupamos em **4 blocos**:

Bloco	Fases originais	Escopo resumido	Status	Commit
1	1-4	Arquitetura/limpeza, banco/segurança, design system, componentes globais e páginas públicas	✅ Concluído	a8dd2db
2	5-7 (+ parte de 8, 10)	Autenticação completa, área do aluno, dashboard, feed social, comunicados do aluno	✅ Concluído	6c768fa
3	9, 11 (+ resto de 8, 10)	Painel admin reconstruído, edição de treinos/comunicados, moderação, gestão de membros	⌚ Pendente	—
4	12-16	PWA, acessibilidade, desempenho, testes, deploy na Vercel	⌚ Pendente	—

O que já foi feito

Bloco 1 — Fundação (Fases 1-4) ✅

Commit: a8dd2db • **Validação:** build/lint/typecheck zero erros

Entrega	Detalhes
Limpeza arquitetural	Eliminada duplicação de actions (<code>app/actions/</code> removido), rotas (<code>/treinos</code> , <code>/perfil</code> , <code>/fotos</code> removidas) e componentes (<code>CreatePost</code> , <code>ProfileForm</code> removidos). <code>lib/actions/</code> é a fonte única.
Banco e segurança	Migration <code>0002_correcoes_rls_fase2.sql</code> : policy DELETE em <code>profiles (admin)</code> , WITH CHECK em update admin. Decisão: manter <code>member / admin</code> (sem <code>treinador</code>); <code>treinos</code> visíveis a todos (decisão de produto documentada).
Design system	Tailwind v4 puro (CSS); <code>tailwind.config.ts</code> removido. Tokens: off-white <code>#F9F7F5</code> , vermelho <code>#DC2626</code> , verde <code>#16A34A</code> . Fontes: Inter (corpo) + Barlow Condensed (títulos). Componentes de UI em <code>components/ui/</code> (Button, Card, Input, Badge, Avatar, Container, Section).
Páginas públicas	Home, Sobre, Galeria, Contato reconstruídas com dados oficiais. Header com rotas reais. Footer consistente. Galeria apenas com fotos reais. Contato via Instagram (sem inventar telefone/email). Rotas legado (<code>/equipe</code> , <code>/historia</code> , <code>/resultados</code>) redirecionam.

Bloco 2 — Autenticação e Área do Aluno (Fases 5-7)

Commit: `6c768fa` • **Validação:** `build/lint/typecheck` zero erros, 23 rotas geradas

Entrega	Detalhes
Autenticação completa	Login com redirecionamento por papel (admin→ /admin , aluno→ /dashboard). reset-Password corrigido (redirectTo usava domínio do Supabase, agora usa origem do site). Nova action updatePassword + página /recuperar-senha/nova . Helper getSiteOrigin() para funcionar em dev e produção.
Dashboard do aluno	Página inicial real (antes era redirect): boas-vindas personalizadas, atalhos rápidos, próximos 3 treinos, últimos 3 comunicados, atividade recente da equipe (4 posts), contagem de publicações próprias. Tudo via Promise.all com 4 queries paralelas.
Comunicados do aluno	Nova rota /dashboard/comunicados com destaque no comunicado mais recente e estado vazio.
Perfil de membros	Nova rota /dashboard/membros/[id] : avatar, bio, cidade, objetivo, badge Treinador (para admin), stats (posts, km somados), últimas 10 atividades. Redireciona para /dashboard/perfil se for o próprio usuário.
Feed social aprimorado	createPost valida conteúdo vazio e foto >10MB com erro visível. Nova action deleteComment (autor ou admin, RLS garante). Post-Card: autor clicável (link para perfil), botão de excluir comentário.
Navegação	DashboardSidebarNav (desktop) + DashboardBottomNav (mobile) com estado ativo por usePathname . 5 itens: Início, Feed, Treinos, Comunicados, Perfil.
Upload de avatar	Validação de 5MB com erro visível (antes falhava silencioso).

O que falta fazer

Bloco 3 — Painel Admin e CRUD Completo (Fases 9, 11) ⌚

Estimativa de créditos: 9.000–13.000

Este bloco foca em tornar o painel administrativo profissional e completar as operações CRUD que hoje só têm create/delete.

3.1 — Reconstrução visual do painel admin

Problema atual: o layout admin (`app/(admin)/layout.tsx`) usa CSS inline fora do design system. As telas admin têm estilo divergente do resto da aplicação.

O que será feito:

- Reconstruir o layout admin usando os tokens e componentes do design system (`components/ui/`).
- Sidebar admin com navegação clara: Dashboard, Treinos, Comunicados, Membros.
- Responsividade mobile (bottom nav ou menu hambúrguer).

Arquivos afetados:

- `app/(admin)/layout.tsx` — reescrita
- Possível criação de `components/admin/AdminNav.tsx`

3.2 — Edição de treinos

Problema atual: `lib/actions/admin.ts` tem `createWorkout` e `deleteWorkout`, mas não tem `updateWorkout`. O treinador não consegue corrigir um treino publicado — precisa excluir e recriar.

O que será feito:

- Nova action `updateWorkout(workoutId, formData)` em `lib/actions/admin.ts`.
- Modal/formulário de edição no painel admin (pré-preenchido com dados existentes).
- Campos: título, descrição, nível (iniciante/intermediário/avançado), objetivo, data agendada.

Arquivos afetados:

- `lib/actions/admin.ts` — nova action
- `app/(admin)/admin/treinos/page.tsx` — botão de editar + modal/formulário
- Possível `components/admin/WorkoutForm.tsx`

3.3 — Edição de comunicados

Problema atual: mesmo cenário dos treinos — só create/delete, sem update.

O que será feito:

- Nova action `updateAnnouncement(announcementId, formData)`.
- Modal/formulário de edição no painel admin.
- Campos: título, conteúdo.

Arquivos afetados:

- `lib/actions/admin.ts` — nova action
- `app/(admin)/admin/comunicados/page.tsx` — botão de editar + modal/formulário

3.4 — Gestão de membros aprimorada

Problema atual: a tela de membros lista e permite promover/rebaixar. A exclusão já funciona (policy adicionada no Bloco 1). Falta: confirmação visual antes de ações destrutivas, feedback de sucesso/erro, e possivelmente visualização do perfil do membro.

O que será feito:

- Modal de confirmação antes de excluir membro ou alterar papel.
- Mensagens de feedback (toast ou inline).
- Link para o perfil público do membro (`/dashboard/membros/[id]`).

Arquivos afetados:

- `app/(admin)/admin/membros/page.tsx`
- Possível `components/ui/ConfirmDialog.tsx`

3.5 — Moderação do feed (admin)

Problema atual: o admin pode excluir posts e comentários via actions, mas não há interface de moderação centralizada.

O que será feito:

- No PostCard: garantir que o admin veja botão de excluir em qualquer post/comentário (já parcialmente feito no Bloco 2 para comentários).
- Avaliar se é necessária uma tela de moderação separada ou se a moderação inline no feed é suficiente.

Arquivos afetados:

- `components/feed/PostCard.tsx` (ajustes pontuais)

Critérios de conclusão do Bloco 3

- [] Build, lint e typecheck: zero erros
- [] Painel admin com design system aplicado (sem CSS inline)
- [] CRUD completo de treinos: criar, editar, excluir
- [] CRUD completo de comunicados: criar, editar, excluir
- [] Gestão de membros com confirmação e feedback
- [] Commit + push para GitHub

Bloco 4 — Qualidade e Deploy (Fases 12-16)

Estimativa de créditos: 7.000–10.000

Este é o bloco final: polimento, qualidade e publicação.

4.1 — PWA e mobile**O que será feito:**

- Revisar `manifest.json` (atalhos apontando para rotas válidas após a reconstrução).
- Verificar theme-color alinhado à paleta (#DC2626 ou #F9F7F5).
- Testar instalabilidade e experiência mobile em 375px.
- Verificar que a navegação inferior do dashboard (feita no Bloco 2) funciona corretamente em contexto PWA.

Arquivos afetados:

- `public/manifest.json`
- `app/layout.tsx` (metadados, viewport)

4.2 — Acessibilidade (WCAG AA)**O que será feito:**

- Auditar contraste de cores em todas as páginas (ferramenta axe ou Lighthouse).
- Garantir `alt` significativo em todas as imagens.
- Foco visível em todos os elementos interativos.
- Labels em todos os inputs de formulário.

- Navegação completa por teclado nos fluxos principais.
- Atributos `aria-*` onde necessário (modais, toasts, navegação).

Arquivos afetados: componentes de UI, páginas públicas e logadas (pontual).

4.3 — Desempenho

O que será feito:

- Otimizar imagens com `next/image` (tamanhos, formatos, priority no LCP).
- Revisar server vs client components (minimizar JavaScript no cliente).
- Pagar feed se necessário (hoje carrega todos os posts).
- Remover domínios desnecessários em `next.config.ts` (ex.: `images.unsplash.com`).
- Verificar e resolver vulnerabilidades de dependências.

Arquivos afetados:

- `next.config.ts`
- Páginas com imagens grandes
- `app/(dashboard)/dashboard/feed/page.tsx` (paginação)

4.4 — Boundaries de erro e estados de carregamento

O que será feito:

- Criar `error.tsx` e `loading.tsx` globais (e por segmento onde necessário).
- Criar `not-found.tsx` global para 404s amigáveis.
- Garantir que erros de servidor não exponham informação sensível.

Arquivos afetados:

- `app/error.tsx`, `app/not-found.tsx`, `app/loading.tsx` (novos)
- Possíveis boundaries por segmento: `app/(dashboard)/error.tsx`, `app/(admin)/error.tsx`

4.5 — Testes

O que será feito:

- Configurar framework de testes (Vitest ou Jest + React Testing Library).
- Testes unitários para server actions (`lib/actions/*`).
- Testes de integração para fluxos de auth.
- Testes e2e dos caminhos críticos (Playwright ou Cypress), se viável com o ambiente.
- Pipeline de CI básico (GitHub Actions) rodando lint + typecheck + build + testes.

Arquivos afetados:

- `vitest.config.ts` ou `jest.config.ts` (novo)
- `__tests__` ou `*.test.ts` (novos)
- `.github/workflows/ci.yml` (novo)

4.6 — Deploy na Vercel

Pré-requisitos (a serem fornecidos pelo usuário):

- Acesso ao projeto Supabase de produção (URL + anon key)
- Acesso ao projeto Vercel (ou conta para criar)
- Domínio desejado (se houver)

O que será feito:

- Configurar variáveis de ambiente na Vercel (`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_SITE_URL`).
- Aplicar migrações no Supabase de produção.
- Definir o treinador Robson como admin no banco (via SQL após primeiro cadastro).

- Validar build de produção.
- Smoke test completo em produção (público + auth + dashboard + admin).
- Atualizar README com instruções de manutenção.

Arquivos afetados:

- README.md (atualização)
- Configuração na Vercel (fora do repo)

Critérios de conclusão do Bloco 4

- ☐ Build, lint e typecheck: zero erros
 - ☐ PWA instalável com atalhos corretos
 - ☐ Lighthouse acessibilidade ≥ 90 nos fluxos principais
 - ☐ Lighthouse performance ≥ 80 em mobile
 - ☐ Boundaries de erro e loading states em todas as rotas
 - ☐ Suíte de testes verde (cobertura mínima dos caminhos críticos)
 - ☐ App publicado e funcional na Vercel
 - ☐ Smoke test de produção aprovado
-

Decisões de projeto tomadas (registro)

#	Decisão	Bloco	Justificativa
1	Papéis: <code>member</code> e <code>admin</code> (sem <code>treinador</code>)	1	Schema real só tem esses dois; simplifica RLS e evita refatoração de CHECK constraints.
2	Treinos visíveis a todos os membros autenticados	1	Coluna <code>assigned_to</code> não existe; equipe treina junta (~200 atletas). Decisão de produto — pode ser revisada com o cliente.
3	Tailwind v4 puro (CSS, sem <code>tailwind.config.ts</code>)	1	Config v3 era código morto no v4; tokens definidos diretamente no CSS.
4	Fontes: Inter (corpo) + Barlow Condensed (títulos)	1	Referências a Outfit removidas; identidade visual consolidada.
5	Contato via Instagram (sem inventar telefone/email)	1	Regra do projeto: nunca fabricar dados de contato.
6	<code>lib/actions/</code> como fonte única de server actions	1	<code>app/actions/</code> estava desalinhada do schema e duplicava lógica.
7	<code>NEXT_PUBLIC_SITE_URL</code> para URLs de redirect (auth)	2	Necessário para que os links de email funcionem em qualquer ambiente (dev/produção).

Riscos conhecidos e pendências

Risco/Pendência	Impacto	Mitigação	Bloco
<code>.env.local</code> com placeholders (sem credenciais reais do Supabase)	Funcionalidades testáveis apenas via build/renderização; sem teste ponta a ponta	Configurar credenciais reais no deploy (Bloco 4)	4
Tipos Database do Supabase não gerados	Erros de coluna só aparecem em runtime, não em compile-time	Gerar tipos quando houver acesso ao Supabase real	4
Painel admin com CSS inline	Visual divergente do design system	Reconstruir no Bloco 3	3
Sem testes automatizados	Regressões só detectáveis manualmente	Implementar no Bloco 4	4
Sem boundaries de erro globais	Erros de servidor mostram tela genérica do Next.js	Implementar no Bloco 4	4
Feed sem paginação	Performance degradada com muitos posts	Implementar no Bloco 4	4
Domínios de imagem desnecessários (<code>unsplash.com</code>) no <code>next.config.ts</code>	Nenhum funcional; vetor de ataque teórico	Remover no Bloco 4	4

Dados imutáveis (regras para qualquer agente)

Estes dados são **oficiais e confirmados**. Nunca devem ser alterados, inventados ou complementados com informação não verificada:

- **Nome:** Born to Run — Treinamento e Saúde
- **Fundação:** 2015
- **Local:** Descalvado-SP
- **Equipe:** ~200 atletas
- **Corridas:** +200 participações
- **Instagram:** @equipeborntorun

- **Treinador:** Robson Alves — Educador Físico, CREF 119911-G/SP, especialização meio fundo/fundo, Treinador nível 1 World Athletics

Proibido:

- Inventar endereço, telefone, CNPJ, depoimentos ou títulos de corrida.
- Guardar credenciais/chaves no código.
- Publicar contatos não confirmados (usar Instagram como canal até que dados reais sejam fornecidos).

Para agentes de IA: como continuar

1. **Leia primeiro:** `AGENTS.md` (regras do projeto), `CURRENT_IMPLEMENTATION_STATUS.md` (status atual), `FABLE_REBUILD_PLAN.md` (plano de 16 fases original).
 2. **Identifique o bloco atual:** veja a tabela no topo deste documento.
 3. **Respeite os critérios de conclusão** de cada bloco antes de marcar como concluído.
 4. **Valide sempre:** `npx tsc --noEmit && npx eslint . && npm run build` — zero erros antes de qualquer commit.
 5. **Commit em português**, referenciando a fase/bloco: ex. `feat(fases-9-11): painel admin reconstruído e CRUD completo`.
 6. **Push para GitHub** ao finalizar cada bloco — o dono do projeto pediu explicitamente.
 7. **Atualize este documento** e o `CURRENT_IMPLEMENTATION_STATUS.md` após cada bloco.
-

Documento temporário — será removido após a conclusão do projeto.